

Chapter 3

Apple II Graphics

Hi-res graphics on the Apple II is odd. Graphics are memory-mapped, not exactly consecutively, and bits don't always correspond to pixels. Color especially is odd, compared to today's luxurious 32-bit per pixel RGBA.

The Apple II has two hi-res graphics pages, and maps the area from \$2000-\$3FFF to high-res graphics page 1 (HGR1), and \$4000-\$5FFF to page 2 (HGR2).

We have routines to clear these screens.

```
5  <routines 5>≡ (281) 26▷
    ORG      $7A51
    CLEAR_HGR1:
    SUBROUTINE

    LDA      #$20          ; Start at $2000
    LDX      #$40          ; End at $4000 (but not including)
    BNE      CLEAR_PAGE    ; Unconditional jump

    CLEAR_HGR2:
    SUBROUTINE

    LDA      #$40          ; Start at $4000
    LDX      #$60          ; End at $6000 (but not including)
    ; fallthrough

    CLEAR_PAGE:
    STA      TMP_PTR+1     ; Start with the page in A.
    LDA      #$00
    STA      TMP_PTR
    TAY
    LDA      #$80          ; fill byte = 0x80

    .loop:
    STA      (TMP_PTR),Y
    INY
    BNE      .loop
```

July 8, 2026

main.nw 6

```
INC    TMP_PTR+1
CPX    TMP_PTR+1
BNE    .loop          ; while TMP_PTR != X * 0x100
RTS
```

Defines:

CLEAR_HGR1, used in chunks 53, 124, and 259.

CLEAR_HGR2, used in chunks 53, 119b, 145, 237, 265, and 268.

Uses TMP_PTR 4.

3.1 Pixels and their color

First we'll talk about pixels. Nominally, the resolution of the hi-res graphics screen is 280 pixels wide by 192 pixels tall. In the memory map, each row is represented by 40 bytes. The high bit of each byte is not used for pixel data, but is used to control color.

Here are some rules for how these bytes are turned into pixels:

- Pixels are drawn to the screen from byte data least significant bit first. This means that for the first byte bit 0 is column 0, bit 1 is column 1, and so on.
- A pattern of 11 results in two white pixels at the 1 positions.
- A pattern of 010 results at least in a colored pixel at the 1 position.
- A pattern of 101 results at least in a colored pixel at the 0 position.
- So, a pattern of 01010 results in at least three consecutive colored pixels starting from the first 1 to the last 1. The last 0 bit would also be colored if followed by a 1.
- Likewise, a pattern of 11011 results in two white pixels, a colored pixel, and then two more white pixels.
- The color of a 010 pixel depends on the column that the 1 falls on, and also whether the high bit of its byte was set or not.
- The color of a 11011 pixel depends on the column that the 0 falls on, and also whether the high bit of its byte was set or not.

	Odd	Even
High bit clear	Green	Violet
High bit set	Orange	Blue

The implication is that you can only select one pair of colors per byte.

An example would probably be good here. We will take one of the sprites from the game.

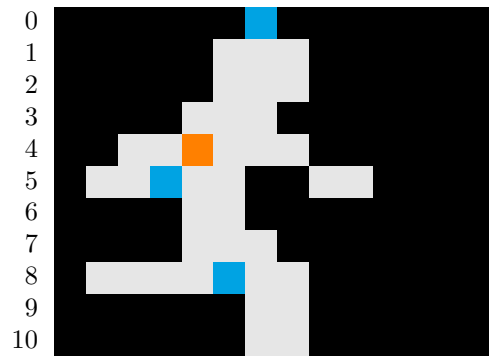
Bytes		Bits		Pixel Data
00	00	0000000	0000000	00000000000000
00	00	0000000	0000000	00000000000000
00	00	0000000	0000000	00000000000000
55	00	1010101	0000000	10101010000000
41	00	1000001	0000000	10000010000000
01	00	0000001	0000000	10000000000000
55	00	1010101	0000000	10101010000000
50	00	1010000	0000000	00001010000000
50	00	1010000	0000000	00001010000000
51	00	1010001	0000000	10001010000000
55	00	1010101	0000000	10101010000000

The game automatically sets the high bit of each byte, so we know we're going to see orange and blue. Assuming that the following bits are all zero, and we place the sprite starting at column 0, we should see this:



Here is a more complex sprite:

Bytes		Bits		Pixel Data
40	00	1000000	0000000	00000010000000
60	01	1100000	0000001	00000111000000
60	01	1100000	0000001	00000111000000
70	00	1110000	0000000	00001110000000
6C	01	1101100	0000001	00110111000000
36	06	0110110	0000110	01101100110000
30	00	0110000	0000000	00001100000000
70	00	1110000	0000000	00001110000000
5E	01	1011110	0000001	01111011000000
40	01	1000000	0000001	00000011000000
40	01	1000000	0000001	00000011000000



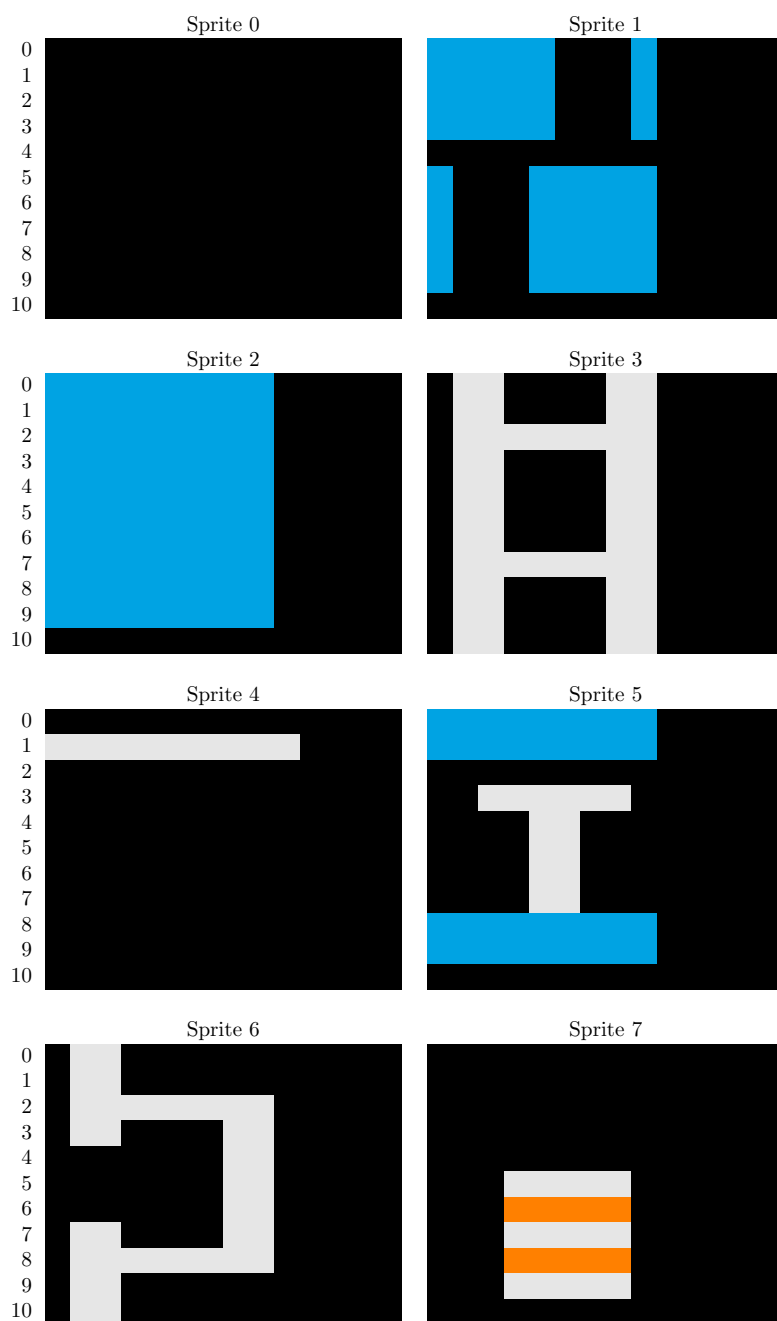
Take note of the orange and blue pixels. All the patterns noted in the rules above are used.

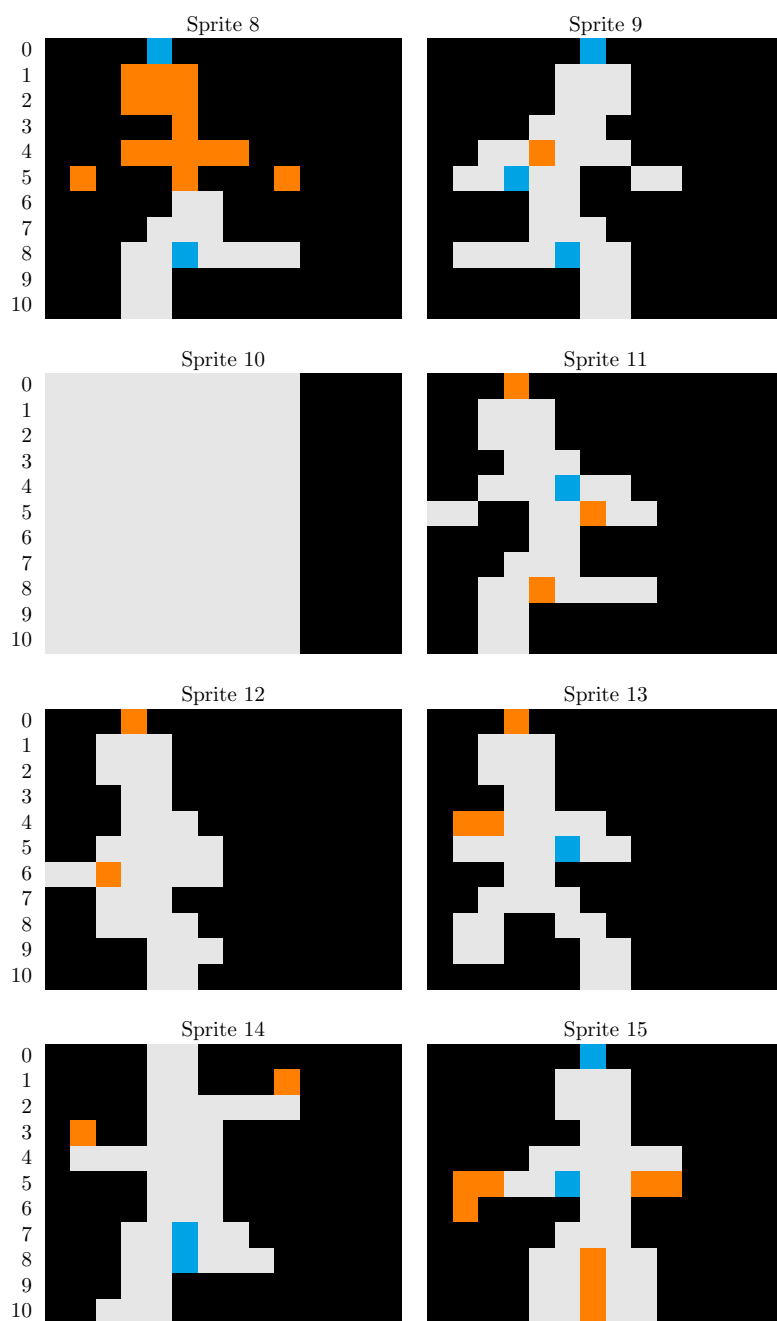
3.2 The sprites

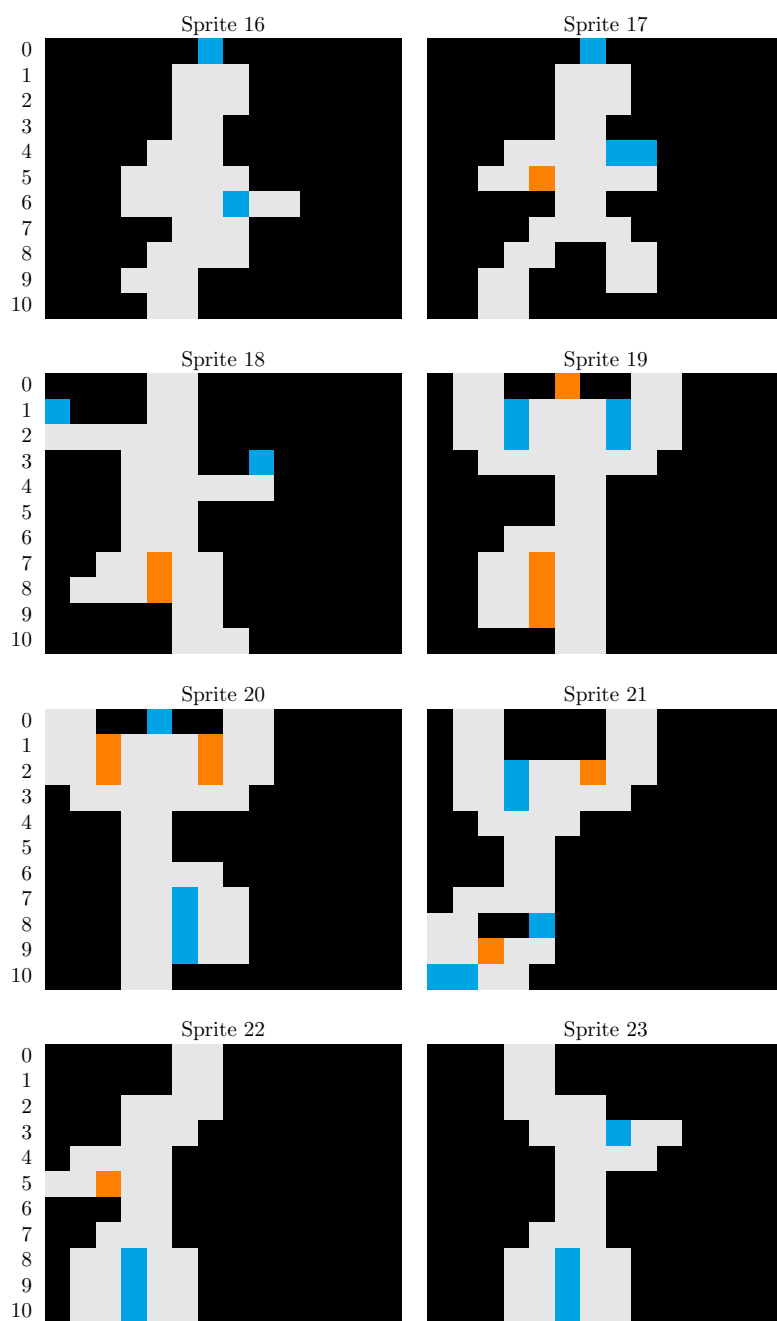
Lode Runner defines 104 sprites, each being 11 rows, with two bytes per row. The first bytes of all 104 sprites are in the table first, then the second bytes, then the third bytes, and so on. Later we will see that only the leftmost 10 pixels out of the 14-pixel description is used.

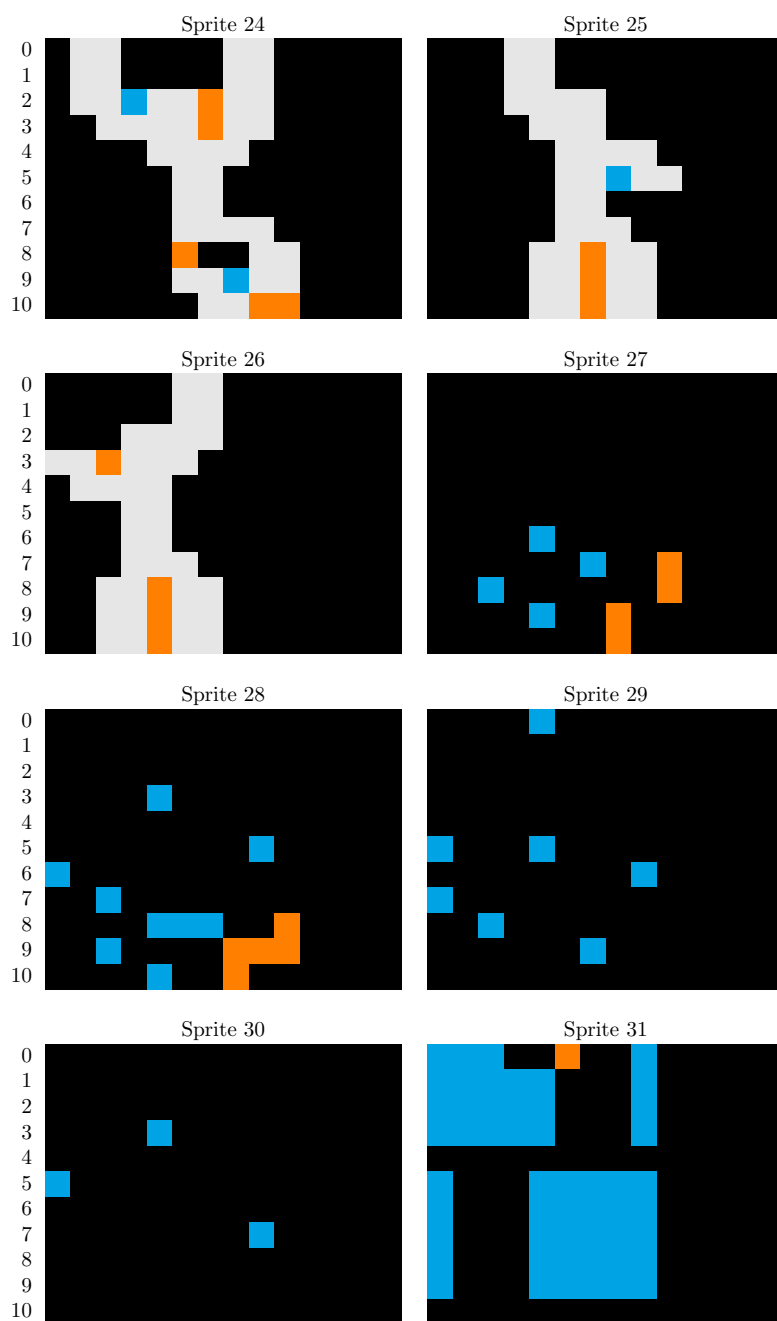
```

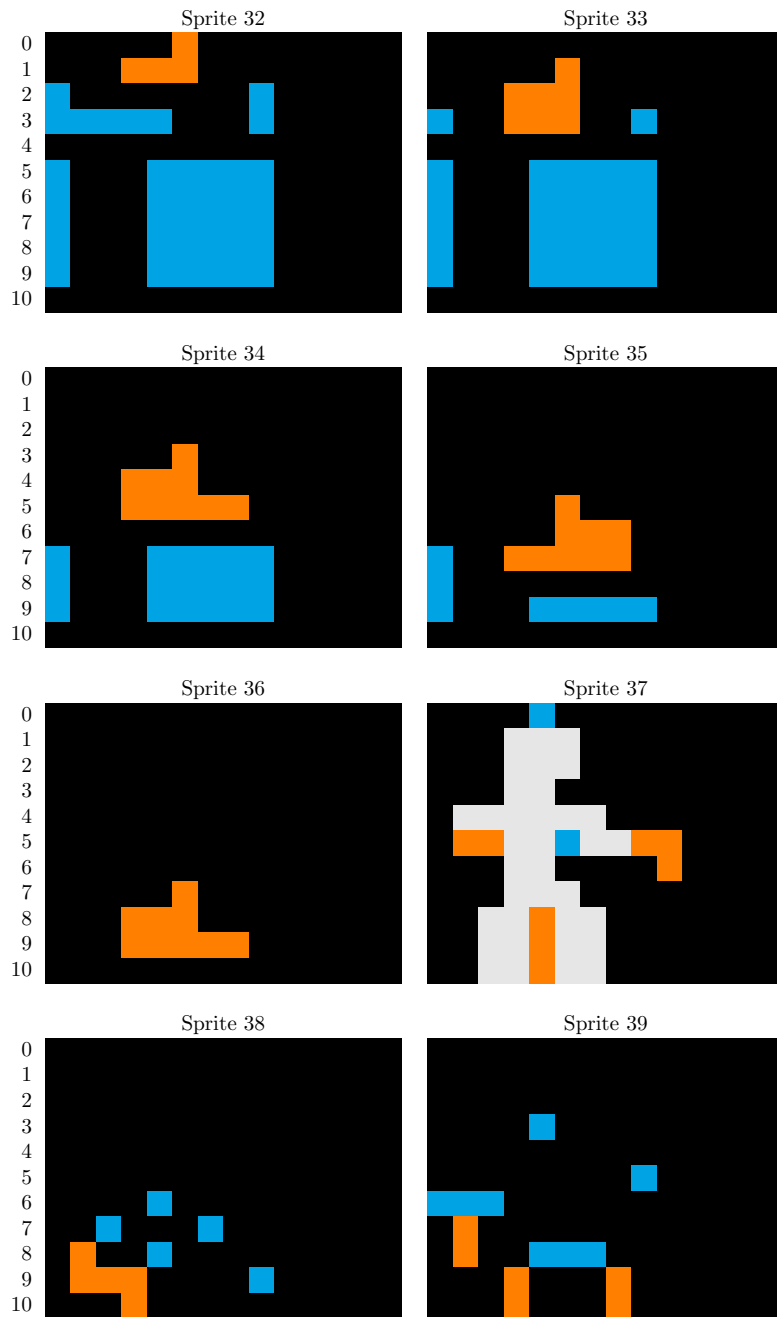
9  <tables 9>≡ (281) 24▷
    ORG      $AD00
    SPRITE_DATA:
        INCLUDE "sprite_data.asm"
Defines:
    SPRITE_DATA, used in chunk 26.
```

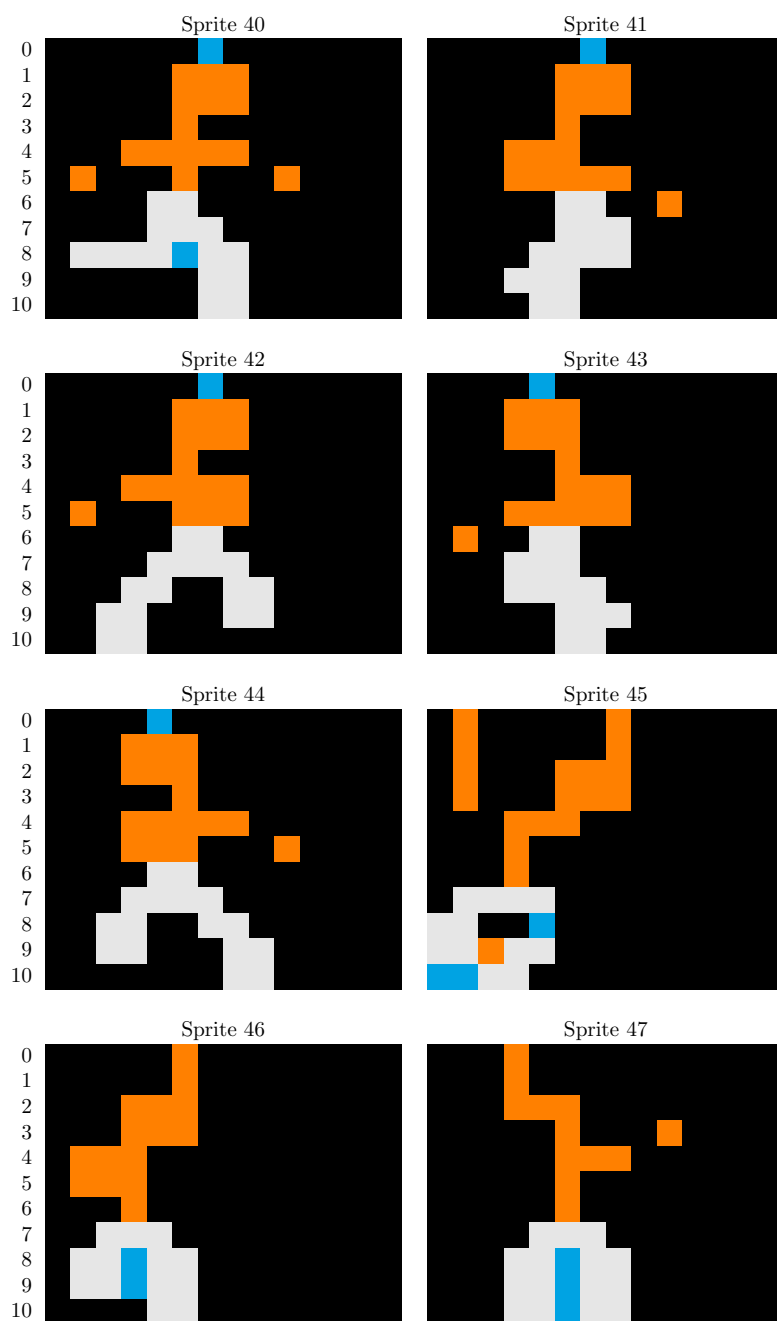


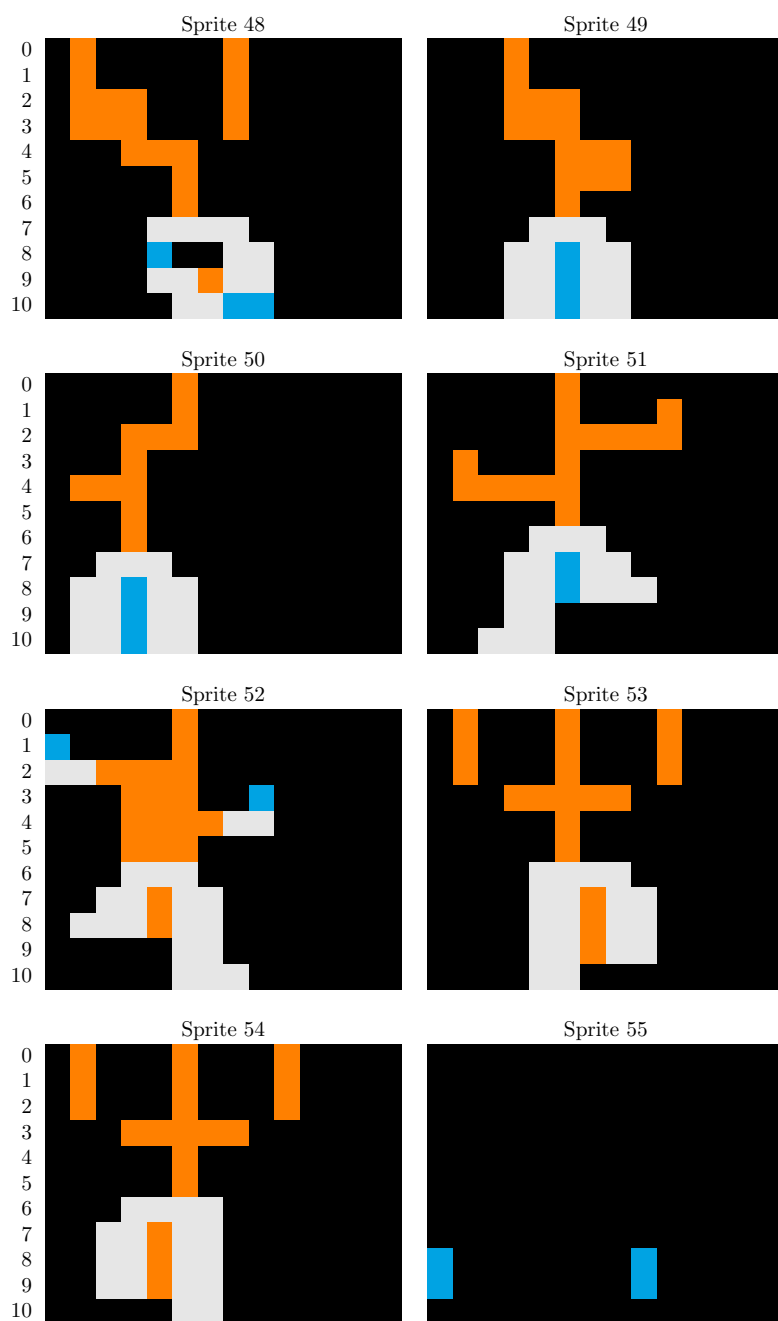


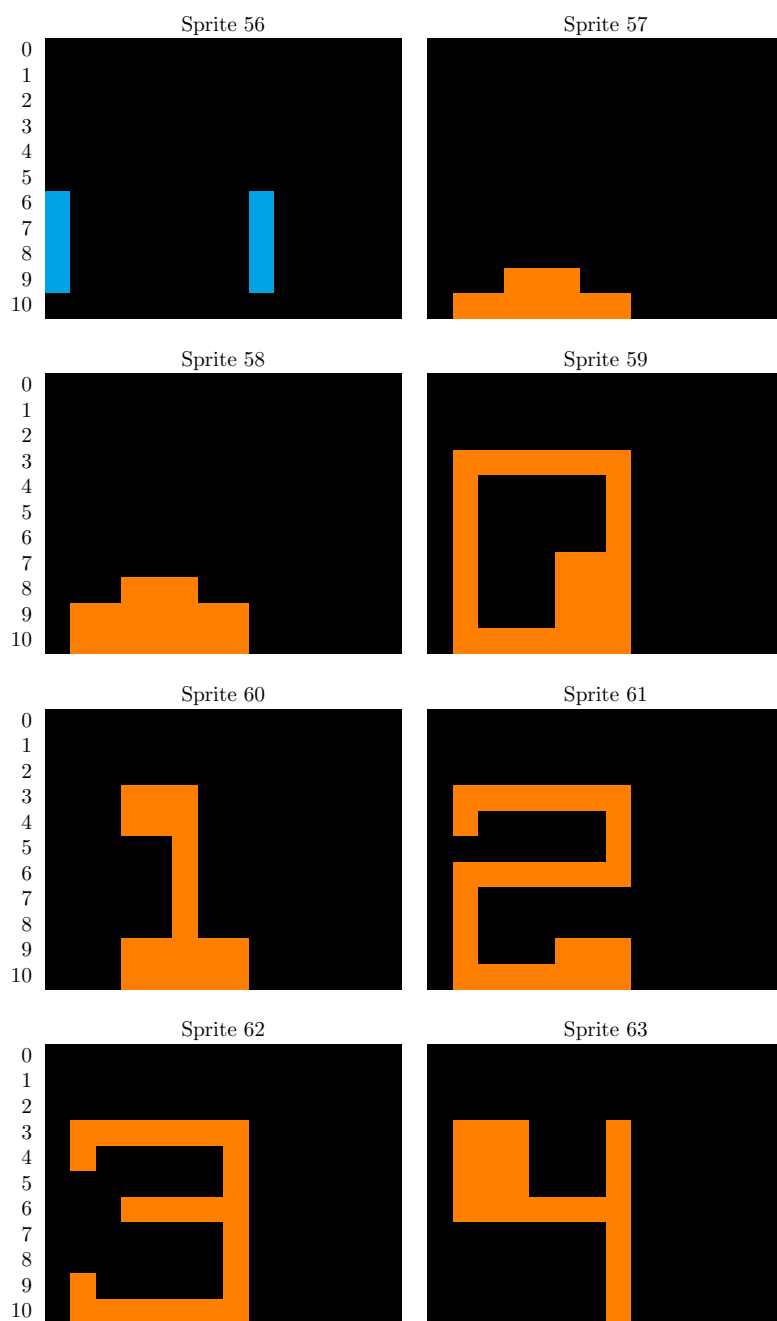


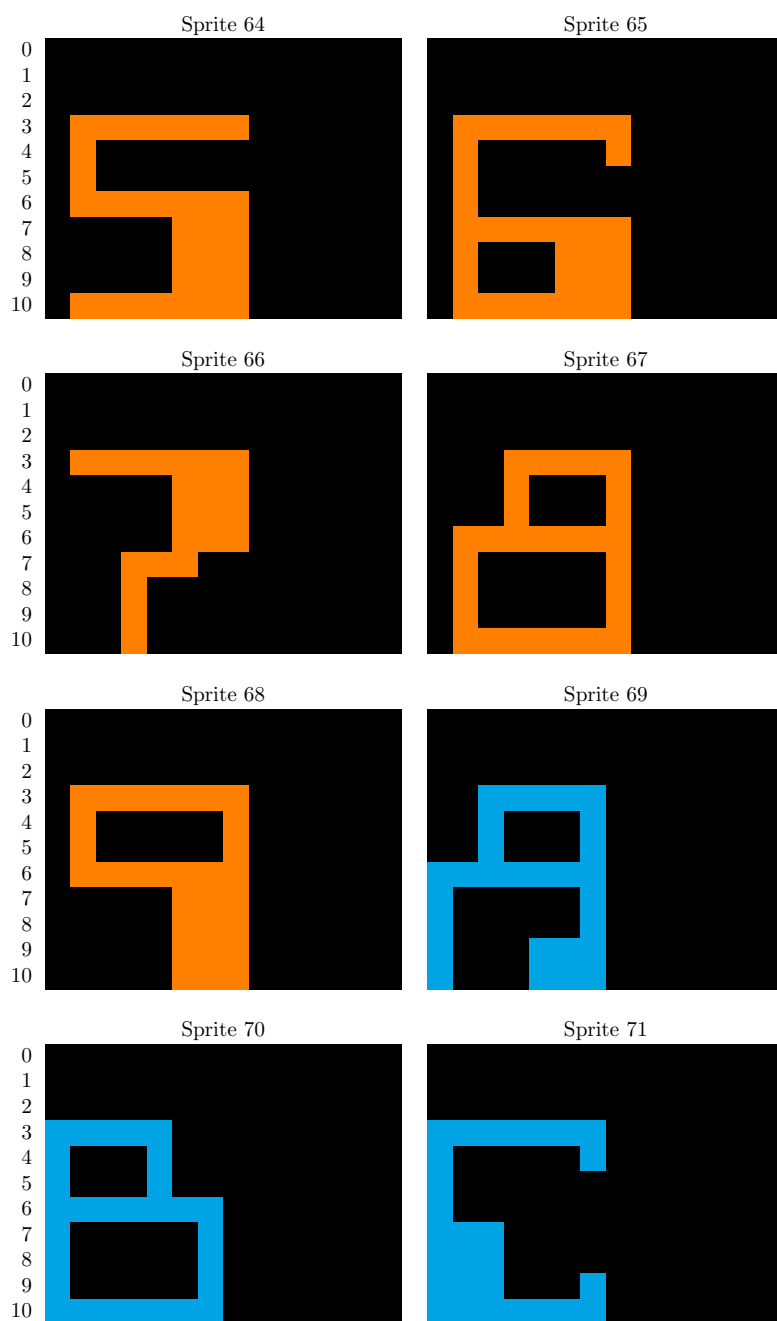


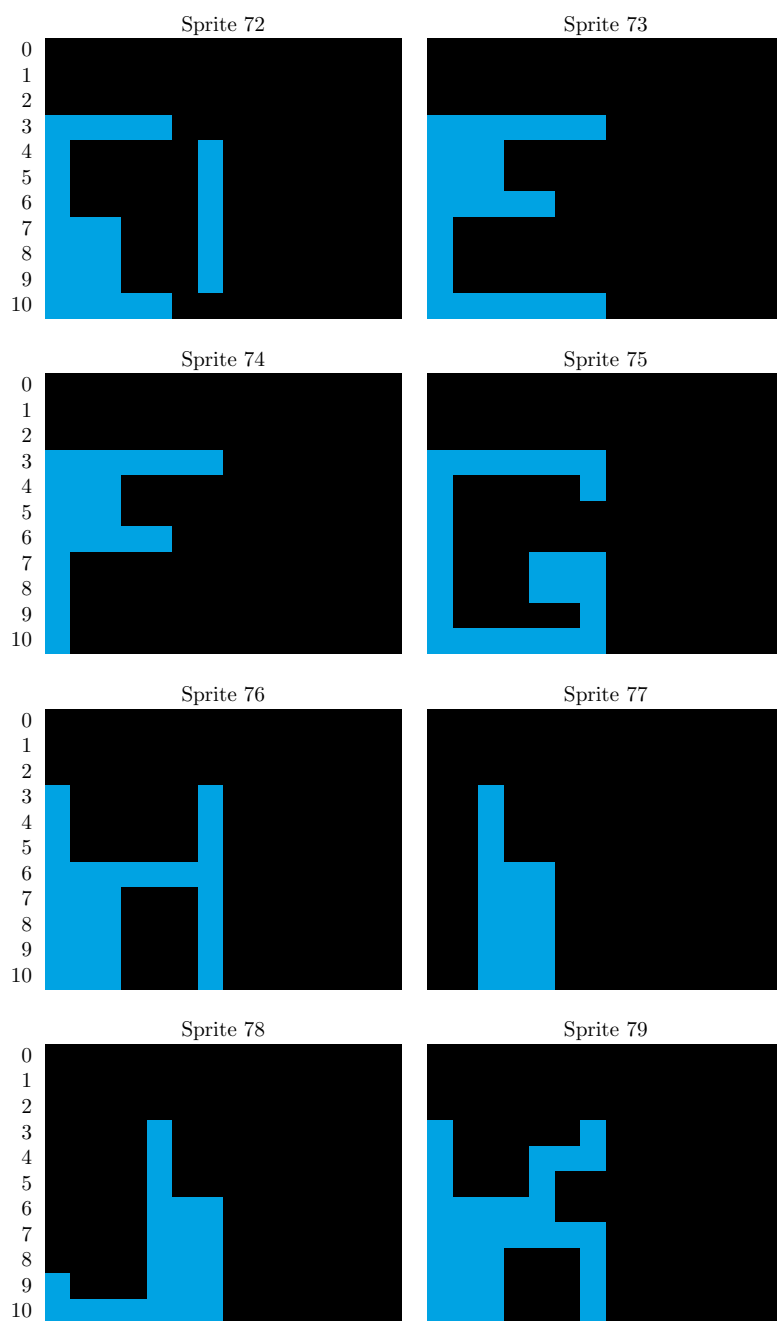


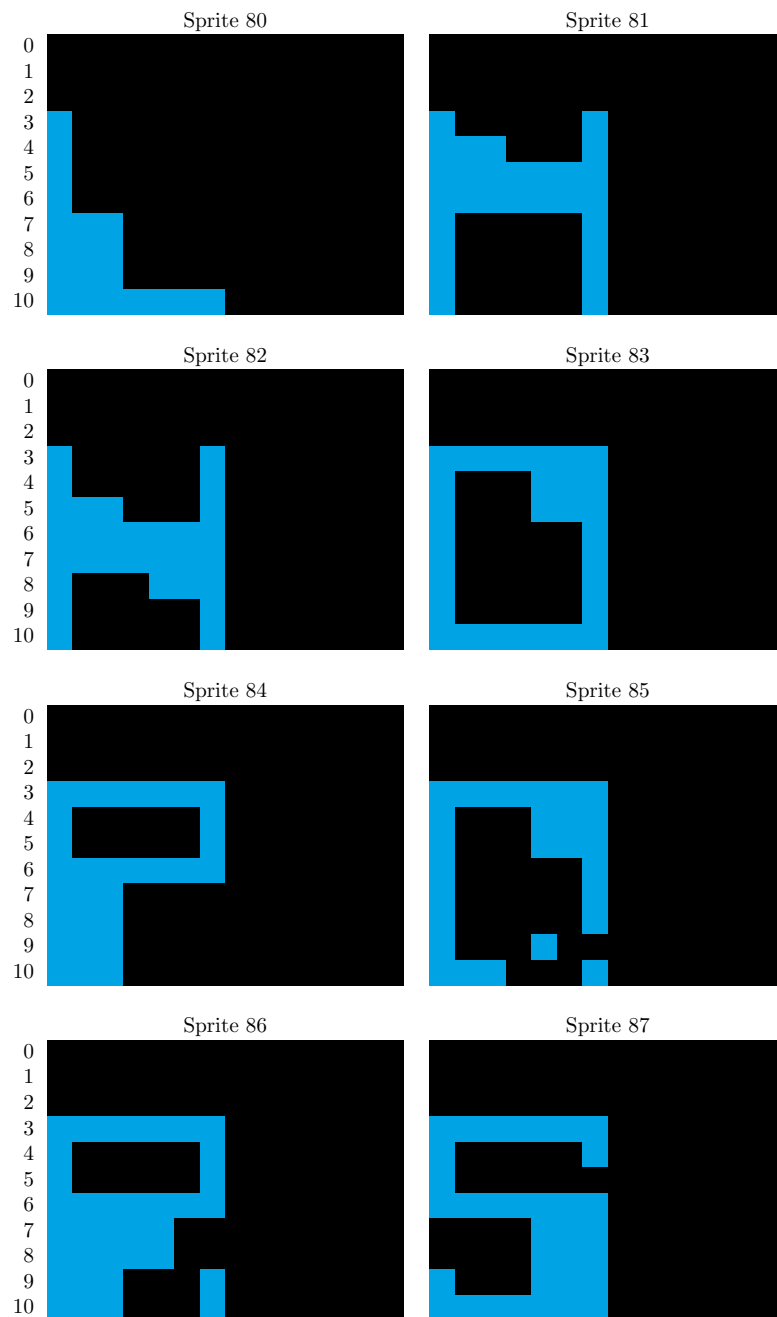


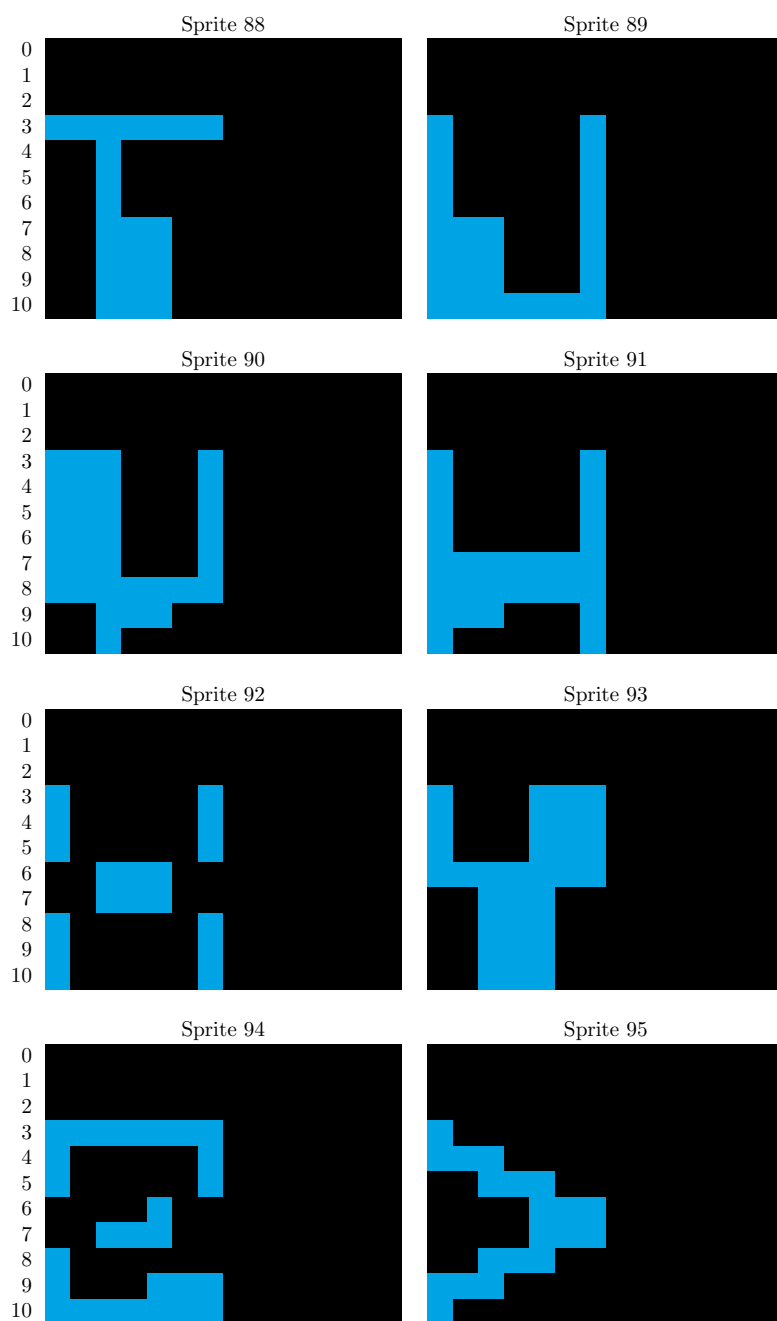


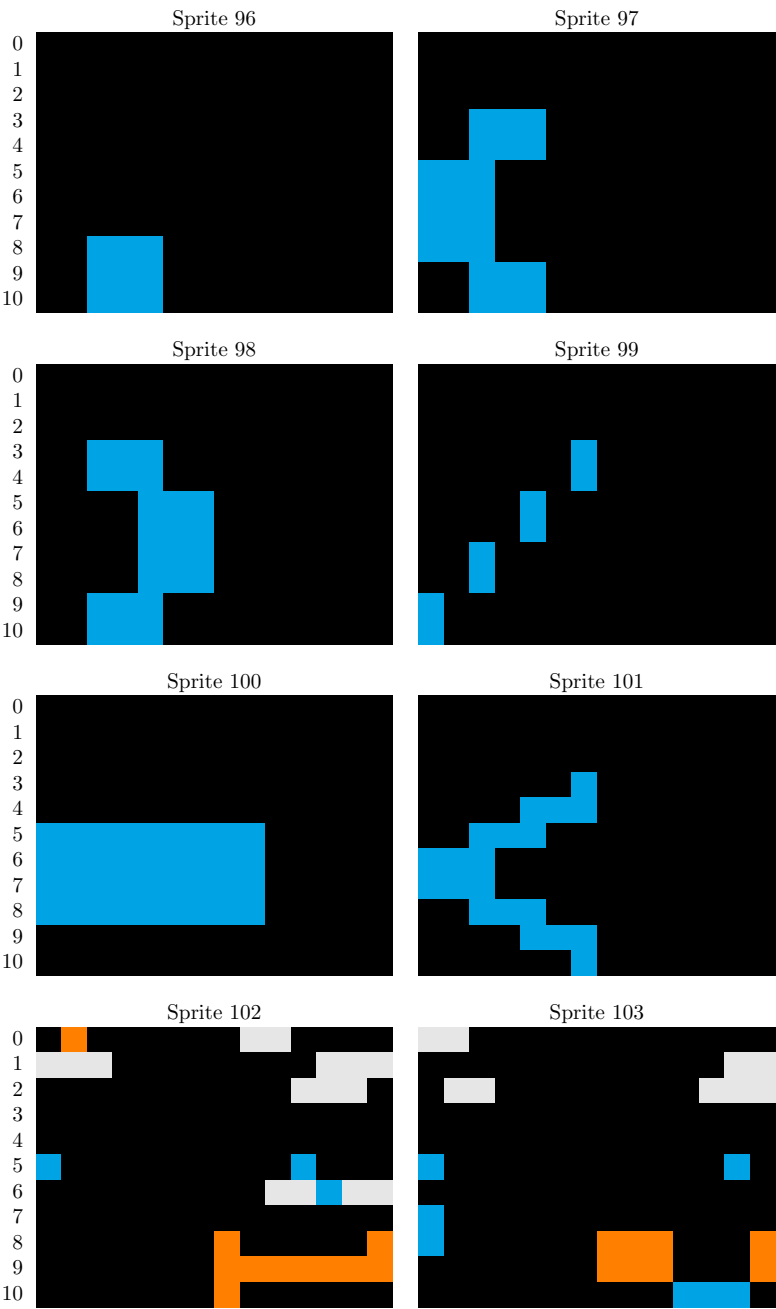












```
22  <defines 4>+=≡ (281) <4 23>
    SPRITE_EMPTY   EQU    #$00
    SPRITE_BRICK    EQU    #$01
    SPRITE_STONE    EQU    #$02
    SPRITE_LADDER    EQU    #$03
```

```
SPRITE_ROPE      EQU    $$04
SPRITE_TRAP      EQU    $$05
SPRITE_INVISIBLE_LADDER EQU    $$06
SPRITE_GOLD      EQU    $$07
SPRITE_GUARD     EQU    $$08
SPRITE_PLAYER    EQU    $$09
SPRITE_ALLWHITE  EQU    $$0A
SPRITE_BRICK_FILLO EQU    $$37
SPRITE_BRICK_FILL1 EQU    $$38
SPRITE_GUARD_EGG0 EQU    $$39
SPRITE_GUARD_EGG1 EQU    $$3A
```

3.3 Shifting sprites

This is all very good if we’re going to draw sprites exactly on 7-pixel boundaries, but what if we want to draw them starting at other columns? In general, such a shifted sprite would straddle three bytes, and Lode Runner sets aside an area of memory at the end of zero page for 11 rows of three bytes that we’ll write to when we want to compute the data for a shifted sprite.

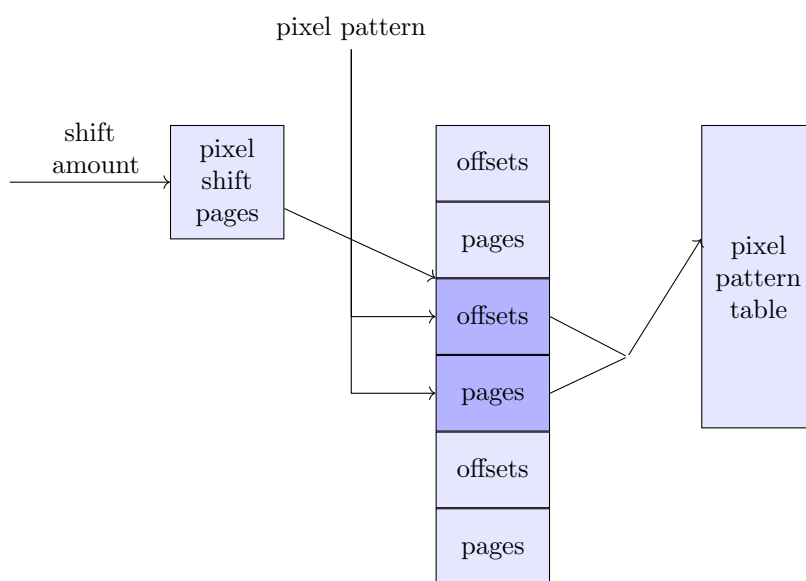
```
23  <defines 4>+≡ (281) <22 25c>
    BLOCK_DATA EQU    $DF ; 33 bytes
```

Defines:
BLOCK_DATA, used in chunks 26, 35, 38, and 41.

Node Runner also contains tables which show how to shift any arbitrary 7-pixel pattern right by any amount from zero to six pixels.

For example, suppose we start with a pixel pattern of 0110001, and we want to shift that right by three bits. The 14-bit result would be 0000110 0010000. However, we have to break that up into bytes, reverse the bits (remember that each byte's bits are output as pixels least significant bit first), and set their high bits, so we end up with 10110000 10000100.

Now, given a shift amount and a pixel pattern, we should be able to find the two-byte shifted pattern. Lode Runner accomplishes this with table lookups as follows:



The pixel pattern table is a table of every possible pattern of 7 consecutive pixels spread out over two bytes. This table is 512 entries, each entry being two bytes. A naive table would have redundancy. For example the pattern 0000100 starting at column 0 is exactly the same as the pattern 0001000 starting at column 1. This table eliminates that redundancy.

```

24      <tables 9>+≡
          ORG      $A900
          PIXEL_PATTERN_TABLE:
          INCLUDE "pixel_pattern_table.asm"
Defines:
          PIXEL_PATTERN_TABLE, never used.

```

Now we just need tables which index into `PIXEL_PATTERN_TABLE` for every 7-pixel pattern and shift value. This table works by having the page number for the shifted pixel pattern at index `shift * 0x100 + 0x80 + pattern` and the offset at index `shift * 0x100 + pattern`.

```
25a  <tables 9>+≡ (281) <24 25b>
      ORG      $A200
      PIXEL_SHIFT_TABLE:
      INCLUDE "pixel_shift_table.asm"
```

Defines:

`PIXEL_SHIFT_TABLE`, never used.

Rather than multiplying the shift value by `0x100`, we instead define another table which holds the page numbers for the shift tables for each shift value.

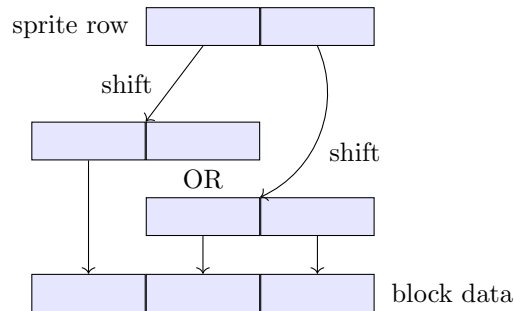
```
25b  <tables 9>+≡ (281) <25a 28a>
      ORG      $84C1
      PIXEL_SHIFT_PAGES:
      HEX      A2 A3 A4 A5 A6 A7 A8
```

Defines:

`PIXEL_SHIFT_PAGES`, used in chunk 26.

So we can get shifted pixels by indexing into all these tables.

Now we can define a routine that will take a sprite number and a pixel shift amount, and write the shifted pixel data into the `BLOCK_DATA` area. The routine first shifts the first byte of the sprite into a two-byte area. Then it shifts the second byte of the sprite, and combines that two-byte result with the first. Thus, we shift two bytes of sprite data into a three-byte result.



Rather than load addresses from the tables and store them, the routine modifies its own instructions with those addresses.

```
25c  <defines 4>+≡ (281) <23 28b>
      ROW_COUNT    EQU    $1D
      SPRITE_NUM    EQU    $1E
```

Defines:

`ROW_COUNT`, used in chunks 26, 35, 38, 41, and 233.

`SPRITE_NUM`, used in chunks 26, 35, 38, 41, 116, 135b, 139, 187b, and 265.

```

26  < routines 5 > + ≡ (281) < 5 28c >
      ORG      $8438
      COMPUTE_SHIFTED_SPRITE:
      SUBROUTINE
      ; Enter routine with X set to pixel shift amount and
      ; SPRITE_NUM containing the sprite number to read.

      .offset_table      EQU $A000          ; Target addresses in read
      .page_table        EQU $A080          ; instructions. The only truly
      .shift_ptr_byte0    EQU $A000          ; necessary value here is the
      .shift_ptr_byte1    EQU $A000          ; 0x80 in .shift_ptr_byte0.

      LDA      #$0B                      ; 11 rows
      STA      ROW_COUNT
      LDA      #<SPRITE_DATA
      STA      TMP_PTR
      LDA      #>SPRITE_DATA
      STA      TMP_PTR+1                  ; TMP_PTR = SPRITE_DATA
      LDA      PIXEL_SHIFT_PAGES,X
      STA      .rd_offset_table + 2
      STA      .rd_page_table + 2
      STA      .rd_offset_table2 + 2
      STA      .rd_page_table2 + 2        ; Fix up pages in lookup instructions
                                          ; based on shift amount (X).

      LDX      #$00                      ; X is the offset into BLOCK_DATA.

      .loop:                            ; === LOOP === (over all 11 rows)
      LDY      SPRITE_NUM
      LDA      (TMP_PTR),Y
      TAY                      ; Get sprite pixel data.

      .rd_offset_table:
      LDA      .offset_table,Y          ; Load offset for shift amount.
      STA      .rd_shift_ptr_byte0 + 1
      CLC
      ADC      #$01
      STA      .rd_shift_ptr_byte1 + 1    ; Fix up instruction offsets with it.

      .rd_page_table:
      LDA      .page_table,Y            ; Load page for shift amount.
      STA      .rd_shift_ptr_byte0 + 2
      STA      .rd_shift_ptr_byte1 + 2    ; Fix up instruction page with it.

      .rd_shift_ptr_byte0:
      LDA      .shift_ptr_byte0          ; Read shifted pixel data byte 0
      STA      BLOCK_DATA,X              ; and store in block data byte 0.

      .rd_shift_ptr_byte1:
      LDA      .shift_ptr_byte1          ; Read shifted pixel data byte 1
      STA      BLOCK_DATA+1,X            ; and store in block data byte 1.

```

```

    LDA    TMP_PTR
    CLC
    ADC    #$68
    STA    TMP_PTR
    LDA    TMP_PTR+1
    ADC    #$00
    STA    TMP_PTR+1                ; TMP_PTR++

    ; Now basically do the same thing with the second sprite byte

    LDY    SPRITE_NUM
    LDA    (TMP_PTR),Y
    TAY                                ; Get sprite pixel data.

.rd_offset_table2:
    LDA    .offset_table,Y           ; Load offset for shift amount.
    STA    .rd_shift_ptr2_byte0 + 1
    CLC
    ADC    #$01
    STA    .rd_shift_ptr2_byte1 + 1  ; Fix up instruction offsets with it.
.rd_page_table2:
    LDA    .page_table,Y            ; Load page for shift amount.
    STA    .rd_shift_ptr2_byte0 + 2
    STA    .rd_shift_ptr2_byte1 + 2 ; Fix up instruction page with it.

.rd_shift_ptr2_byte0:
    LDA    .shift_ptr_byte0         ; Read shifted pixel data byte 0
    ORA    BLOCK_DATA+1,X           ; OR with previous block data byte 1
    STA    BLOCK_DATA+1,X           ; and store in block data byte 1.
.rd_shift_ptr2_byte1:
    LDA    .shift_ptr_byte1         ; Read shifted pixel data byte 1
    STA    BLOCK_DATA+2,X           ; and store in block data byte 2.

    LDA    TMP_PTR
    CLC
    ADC    #$68
    STA    TMP_PTR
    LDA    TMP_PTR+1
    ADC    #$00
    STA    TMP_PTR+1                ; TMP_PTR++

    INX
    INX
    INX                                ; X += 3
    DEC    ROW_COUNT                ; ROW_COUNT--
    BNE    .loop                    ; loop while ROW_COUNT > 0
    RTS

```

Defines:

COMPUTE_SHIFTED_SPRITE, used in chunks 35, 38, and 41.

Uses BLOCK_DATA 23, PIXEL_SHIFT_PAGES 25b, ROW_COUNT 25c, SPRITE_DATA 9, SPRITE_NUM 25c,

and TMP_PTR 4.

3.4 Memory mapped graphics

Within a screen row, consecutive bytes map to consecutive pixels. However, rows themselves are not consecutive in memory.

To make it easy to convert a row number from 0 to 191 to a base address, Lode Runner has a table and a routine to use that table.

```
28a  <tables 9>+≡ (281) <25b 30>
      ORG      $1A85
      ROW_TO_OFFSET_LO:
      INCLUDE "row_to_offset_lo_table.asm"
      ROW_TO_OFFSET_HI:
      INCLUDE "row_to_offset_hi_table.asm"
```

Defines:

ROW_TO_OFFSET_HI, used in chunks 28c and 29a.

ROW_TO_OFFSET_LO, used in chunks 28c and 29a.

```
28b  <defines 4>+≡ (281) <25c 29b>
      ROW_ADDR      EQU      $0C      ; 2 bytes
      ROW_ADDR2     EQU      $0E      ; 2 bytes
      HGR_PAGE      EQU      $1F      ; 0x20 for HGR1, 0x40 for HGR2
```

Defines:

HGR_PAGE, used in chunks 28c, 35, 124, and 254.

ROW_ADDR, used in chunks 28c, 29a, 35, 38, 41, 86, 98, 109, 125, and 256.

ROW_ADDR2, used in chunks 29a, 38, 41, 86, and 98.

```
28c  <routines 5>+≡ (281) <26 29a>
      ORG      $7A31
      ROW_TO_ADDR:
      SUBROUTINE
      ; Enter routine with Y set to row. Base address
      ; (for column 0) will be placed in ROW_ADDR.

      LDA      ROW_TO_OFFSET_LO,Y
      STA      ROW_ADDR
      LDA      ROW_TO_OFFSET_HI,Y
      ORA      HGR_PAGE
      STA      ROW_ADDR+1
      RTS
```

Defines:

ROW_TO_ADDR, used in chunks 35, 125, and 256.

Uses HGR_PAGE 28b, ROW_ADDR 28b, ROW_TO_OFFSET_HI 28a, and ROW_TO_OFFSET_LO 28a.

There's also a routine to load the address for both page 1 and page 2.

```

29a  < routines 5 > +≡ (281) < 28c 31a >
      ORG      $7A3E
      ROW_TO_ADDR_FOR_BOTH_PAGES:
      SUBROUTINE
      ; Enter routine with Y set to row. Base address
      ; (for column 0) will be placed in ROW_ADDR (for page 1)
      ; and ROW_ADDR2 (for page 2).

      LDA      ROW_TO_OFFSET_LO,Y
      STA      ROW_ADDR
      STA      ROW_ADDR2
      LDA      ROW_TO_OFFSET_HI,Y
      ORA      #$20
      STA      ROW_ADDR+1
      EOR      #$60
      STA      ROW_ADDR2+1
      RTS

```

Defines:

ROW_TO_ADDR_FOR_BOTH_PAGES, used in chunks 38, 41, and 94–97.

Uses ROW_ADDR 28b, ROW_ADDR2 28b, ROW_TO_OFFSET_HI 28a, and ROW_TO_OFFSET_LO 28a.

Lode Runner's screens are organized into 28 sprites across by 17 sprites down. To convert between sprite coordinates and screen coordinates and vice-versa, we use tables and lookup routines. Each sprite is 10 pixels across by 11 pixels down.

Note that the last row is used for the status, so actually the game screen is 16 sprites vertically.

```

29b  < defines 4 > +≡ (281) < 28b 34a >
      MAX_GAME_COL      EQU      #27      ; 0x1B
      MAX_GAME_ROW      EQU      #15      ; 0x0F

```

```

30  <tables 9>+= (281) <28a 32b>
      ORG      $1C35
      HALF_SCREEN_COL_TABLE:
          ; 28 cols of 5 double-pixels each
          HEX    00 05 0a 0f 14 19 1e 23 28 2d 32 37 3c 41 46 4b
          HEX    50 55 5a 5f 64 69 6e 73 78 7d 82 87
      SCREEN_ROW_TABLE:
          ; 17 rows of 11 pixels each
          HEX    00 0B 16 21 2C 37 42 4D 58 63 6E 79 84 8F 9A A5
          HEX    B5
      COL_BYTE_TABLE:
          ; Byte number
          HEX    00 01 02 04 05 07 08 0A 0B 0C 0E 0F 11 12 14 15
          HEX    16 18 19 1B 1C 1E 1F 20 22 23 25 26
      COL_SHIFT_TABLE:
          ; Right shift amount
          HEX    00 03 06 02 05 01 04 00 03 06 02 05 01 04 00 03
          HEX    06 02 05 01 04 00 03 06 02 05 01 04
      HALF_SCREEN_COL_BYTE_TABLE:
          HEX    00 00 00 00 01 01 01 02 02 02 02 03 03 03 04 04
          HEX    04 04 05 05 05 06 06 06 06 07 07 07 08 08 08 08
          HEX    09 09 09 0A 0A 0A 0A 0B 0B 0B 0C 0C 0C 0C 0D 0D
          HEX    0D 0E 0E 0E 0E 0F 0F 0F 10 10 10 10 11 11 11 12
          HEX    12 12 12 13 13 13 14 14 14 14 15 15 15 16 16 16
          HEX    16 17 17 17 18 18 18 18 19 19 19 1A 1A 1A 1A 1B
          HEX    1B 1B 1C 1C 1C 1C 1D 1D 1D 1E 1E 1E 1E 1F 1F 1F
          HEX    20 20 20 20 21 21 21 22 22 22 22 23 23 23 24 24
          HEX    24 24 25 25 25 26 26 26 26 27 27 27
      HALF_SCREEN_COL_SHIFT_TABLE:
          HEX    00 02 04 06 01 03 05 00 02 04 06 01 03 05 00 02
          HEX    04 06 01 03 05 00 02 04 06 01 03 05 00 02 04 06
          HEX    01 03 05 00 02 04 06 01 03 05 00 02 04 06 01 03
          HEX    05 00 02 04 06 01 03 05 00 02 04 06 01 03 05 00
          HEX    02 04 06 01 03 05 00 02 04 06 01 03 05 00 02 04
          HEX    06 01 03 05 00 02 04 06 01 03 05 00 02 04 06 01
          HEX    03 05 00 02 04 06 01 03 05 00 02 04 06 01 03 05
          HEX    00 02 04 06 01 03 05 00 02 04 06 01 03 05 00 02
          HEX    04 06 01 03 05 00 02 04 06 01 03 05

```

Defines:

```

COL_BYTE_TABLE, used in chunks 31b and 35.
COL_SHIFT_TABLE, used in chunks 31b and 35.
HALF_SCREEN_COL_BYTE_TABLE, used in chunk 32a.
HALF_SCREEN_COL_SHIFT_TABLE, used in chunk 32a.
HALF_SCREEN_COL_TABLE, used in chunk 31a.
SCREEN_ROW_TABLE, used in chunks 31a and 35.

```

Here is the routine to return the screen coordinates for the given sprite coordinates. The reason that `GET_SCREEN_COORDS_FOR` returns half the screen column coordinate is that otherwise the screen column coordinate wouldn't fit in a register.

```

31a  < routines 5 > +≡ (281) <29a 31b>
      ORG      $885D
      GET_SCREEN_COORDS_FOR:
      SUBROUTINE
      ; Enter routine with Y set to sprite row (0-16) and
      ; X set to sprite column (0-27). On return, Y will be set to
      ; screen row, and X is set to half screen column.

      LDA      SCREEN_ROW_TABLE,Y
      PHA
      LDA      HALF_SCREEN_COL_TABLE,X
      TAX                      ; X = HALF_SCREEN_COL_TABLE[X]
      PLA
      TAY                      ; Y = SCREEN_ROW_TABLE[Y]
      RTS

```

Defines:

`GET_SCREEN_COORDS_FOR`, used in chunks 33, 35, 126, 137, 168, 171, 179, 184, 193, and 212.
 Uses `HALF_SCREEN_COL_TABLE` 30 and `SCREEN_ROW_TABLE` 30.

This routine takes a sprite column and converts it to the memory-mapped byte offset and right-shift amount.

```

31b  < routines 5 > +≡ (281) <31a 32a>
      ORG      $8868
      GET_BYTE_AND_SHIFT_FOR_COL:
      SUBROUTINE
      ; Enter routine with X set to sprite column. On
      ; return, A will be set to screen column byte number
      ; and X will be set to an additional right shift amount.

      LDA      COL_BYTE_TABLE,X
      PHA                      ; A = COL_BYTE_TABLE[X]
      LDA      COL_SHIFT_TABLE,X
      TAX                      ; X = COL_SHIFT_TABLE[X]
      PLA
      RTS

```

Defines:

`GET_BYTE_AND_SHIFT_FOR_COL`, used in chunk 35.
 Uses `COL_BYTE_TABLE` 30 and `COL_SHIFT_TABLE` 30.

This routine takes half the screen column coordinate and converts it to the memory-mapped byte offset and right-shift amount.

```

32a  < routines 5 > + ≡ (281) < 31b 33a >
      ORG      $8872
      GET_BYTE_AND_SHIFT_FOR_HALF_SCREEN_COL:
      SUBROUTINE
      ; Enter routine with X set to half screen column. On
      ; return, A will be set to screen column byte number
      ; and X will be set to an additional right shift amount.

      LDA      HALF_SCREEN_COL_BYTE_TABLE, X
      PHA                      ; A = HALF_SCREEN_COL_BYTE_TABLE[X]
      LDA      HALF_SCREEN_COL_SHIFT_TABLE, X
      TAX                      ; X = HALF_SCREEN_COL_SHIFT_TABLE[X]
      PLA
      RTS

```

Defines:

GET_BYTE_AND_SHIFT_FOR_HALF_SCREEN_COL, used in chunks 38 and 41.

Uses HALF_SCREEN_COL_BYTE_TABLE 30 and HALF_SCREEN_COL_SHIFT_TABLE 30.

We also have some utility routines that let us take a sprite row or column and get its screen row or half column, but offset in either row or column by anywhere from -2 to +2.

```

32b  < tables 9 > + ≡ (281) < 30 33b >
      ORG      $888A
      ROW_OFFSET_TABLE:
      HEX      FB FD 00 02 04

```

Defines:

ROW_OFFSET_TABLE, used in chunk 33a.

33a $\langle routines\ 5 \rangle + \equiv$ (281) $\langle 32a\ 33c \rangle$

```

    ORG      $887C
GET_SCREEN_ROW_OFFSET_IN_X_FOR:
    SUBROUTINE
    ; Enter routine with X set to offset+2 (in double-pixels) and
    ; Y set to sprite row. On return, X will retain its value and
    ; Y will be set to the screen row.

    TXA
    PHA
    JSR      GET_SCREEN_COORDS_FOR
    PLA
    TAX                      ; Restore X
    TYA
    CLC
    ADC      ROW_OFFSET_TABLE,X
    TAY
    RTS

```

Defines:

GET_SCREEN_ROW_OFFSET_IN_X_FOR, used in chunks 135b and 187b.

Uses GET_SCREEN_COORDS_FOR 31a and ROW_OFFSET_TABLE 32b.

33b $\langle tables\ 9 \rangle + \equiv$ (281) $\langle 32b\ 34b \rangle$

```

    ORG      $889D
COL_OFFSET_TABLE:
    HEX      FE FF 00 01 02

```

Defines:

COL_OFFSET_TABLE, used in chunk 33c.

33c $\langle routines\ 5 \rangle + \equiv$ (281) $\langle 33a\ 35 \rangle$

```

    ORG      $888F
GET_HALF_SCREEN_COL_OFFSET_IN_Y_FOR:
    SUBROUTINE
    ; Enter routine with Y set to offset+2 (in double-pixels) and
    ; X set to sprite column. On return, Y will retain its value and
    ; X will be set to the half screen column.

    TYA
    PHA
    JSR      GET_SCREEN_COORDS_FOR
    PLA
    TAY                      ; Restore Y
    TXA
    CLC
    ADC      COL_OFFSET_TABLE,Y
    TAX
    RTS

```

Defines:

GET_HALF_SCREEN_COL_OFFSET_IN_Y_FOR, used in chunks 135b and 187b.

Uses COL_OFFSET_TABLE 33b and GET_SCREEN_COORDS_FOR 31a.

Now we can finally write the routines that draw a sprite on the screen. We have one routine that draws a sprite at a given game row and game column. There are two entry points, one to draw on HGR1, and one for HGR2.

```
34a  <defines 4>+≡ (281) <29b 40>
      ROWNUM      EQU      $1B
      COLNUM      EQU      $1C
      MASK0       EQU      $50
      MASK1       EQU      $51
      COL_SHIFT_AMT EQU      $71
      GAME_COLNUM EQU      $85
      GAME_ROWNUM EQU      $86
```

Defines:

COLNUM, used in chunks 35, 38, and 41.

COL_SHIFT_AMT, used in chunks 35, 38, and 41.

GAME_COLNUM, used in chunks 35, 46a, 48a, 51, 53, 74, 81b, 85b, 87, 114, 116, 119b, 126, 137, 145, 168, 171, 174, 179, 184, 188, 193, 212, 233, 237, 250, 259, 265, 266, and 268.

GAME_ROWNUM, used in chunks 35, 46a, 51, 53, 77, 82-85, 87, 112b, 113, 115a, 116, 119b, 124-26, 131a, 132a, 134d, 137, 145, 168, 171, 174, 179, 184, 188, 193, 212, 233, 237, 250, 256, 259, 265, 266, and 268.

MASK0, used in chunks 35 and 231.

MASK1, used in chunk 35.

ROWNUM, used in chunks 35, 38, 41, and 201.

```
34b  <tables 9>+≡ (281) <33b 60b>
      ORG      $8328
      PIXEL_MASK0:
      BYTE     %00000000
      BYTE     %00000001
      BYTE     %00000011
      BYTE     %00000111
      BYTE     %00001111
      BYTE     %00011111
      BYTE     %00111111
      PIXEL_MASK1:
      BYTE     %11111000
      BYTE     %11111000
      BYTE     %11100000
      BYTE     %11000000
      BYTE     %10000000
      BYTE     %11111110
      BYTE     %11111100
```

Defines:

PIXEL_MASK0, used in chunk 35.

PIXEL_MASK1, used in chunk 35.

```

35  < routines 5 > +≡ (281) < 33c 90 >
      ORG      $82AA
DRAW_SPRITE_PAGE1:
      SUBROUTINE
      ; Enter routine with A set to sprite number to draw,
      ; GAME_ROWNUM set to the row to draw it at, and GAME_COLNUM
      ; set to the column to draw it at.

      STA      SPRITE_NUM
      LDA      #$20          ; Page number for HGR1
      BNE      DRAW_SPRITE   ; Actually unconditional jump

DRAW_SPRITE_PAGE2:
      SUBROUTINE
      ; Enter routine with A set to sprite number to draw,
      ; GAME_ROWNUM set to the row to draw it at, and GAME_COLNUM
      ; set to the column to draw it at.

      STA      SPRITE_NUM
      LDA      #$40          ; Page number for HGR2
      ; fallthrough

DRAW_SPRITE:
      STA      HGR_PAGE
      LDY      GAME_ROWNUM
      JSR      GET_SCREEN_COORDS_FOR
      STY      ROWNUM        ; ROWNUM = SCREEN_ROW_TABLE[GAME_ROWNUM]

      LDX      GAME_COLNUM
      JSR      GET_BYTE_AND_SHIFT_FOR_COL
      STA      COLNUM        ; COLNUM = COL_BYTE_TABLE[GAME_COLNUM]
      STX      COL_SHIFT_AMT ; COL_SHIFT_AMT = COL_SHIFT_TABLE[GAME_COLNUM]

      LDA      PIXEL_MASK0,X
      STA      MASK0        ; MASK0 = PIXEL_MASK0[COL_SHIFT_AMT]
      LDA      PIXEL_MASK1,X
      STA      MASK1        ; MASK1 = PIXEL_MASK1[COL_SHIFT_AMT]

      JSR      COMPUTE_SHIFTED_SPRITE

      LDA      #$0B
      STA      ROW_COUNT
      LDX      #$00
      LDA      COL_SHIFT_AMT
      CMP      #$05
      BCS      .need_3_bytes ; If COL_SHIFT_AMT >= 5, we need to alter three
                           ; screen bytes, otherwise just two bytes.

.loop1:
      LDY      ROWNUM

```

```

JSR    ROW_TO_ADDR
LDY    COLNUM
LDA    (ROW_ADDR),Y
AND    MASK0
ORA    BLOCK_DATA,X
STA    (ROW_ADDR),Y      ; screen[COLNUM] =
                          ;   screen[COLNUM] & MASK0 | BLOCK_DATA[i]

INX
INY
LDY    (ROW_ADDR),Y
AND    MASK1
ORA    BLOCK_DATA,X
STA    (ROW_ADDR),Y      ; screen[COLNUM+1] =
                          ;   screen[COLNUM+1] & MASK1 | BLOCK_DATA[i+1]

INX
INX
INC    ROWNUM
DEC    ROW_COUNT
BNE    .loop1
RTS

; X += 2
; ROWNUM++
; ROW_COUNT--
; loop while ROW_COUNT > 0

.need_3_bytes
LDY    ROWNUM
JSR    ROW_TO_ADDR
LDY    COLNUM
LDA    (ROW_ADDR),Y
AND    MASK0
ORA    BLOCK_DATA,X
STA    (ROW_ADDR),Y      ; screen[COLNUM] =
                          ;   screen[COLNUM] & MASK0 | BLOCK_DATA[i]

INX
INY
LDY    BLOCK_DATA,X
STA    (ROW_ADDR),Y      ; screen[COLNUM+1] = BLOCK_DATA[i+1]

INX
INY
LDY    (ROW_ADDR),Y
AND    MASK1
ORA    BLOCK_DATA,X
STA    (ROW_ADDR),Y      ; screen[COLNUM+2] =
                          ;   screen[COLNUM+2] & MASK1 | BLOCK_DATA[i+2]

INX
INC    ROWNUM
DEC    ROW_COUNT
BNE    .need_3_bytes
; X++
; ROWNUM++
; ROW_COUNT--
; loop while ROW_COUNT > 0

```


RTS

Defines:

DRAW_SPRITE_PAGE1, used in chunks 46a, 48a, 71, 126, 168, 171, 174, 188, and 265.

DRAW_SPRITE_PAGE2, used in chunks 46a, 48a, 70, 85b, 87, 126, 137, 145, 174, 179, 184, 188, 193, and 212.

Uses BLOCK_DATA 23, COLNUM 34a, COL_BYTE_TABLE 30, COL_SHIFT_AMT 34a,

COL_SHIFT_TABLE 30, COMPUTE_SHIFTED_SPRITE 26, GAME_COLNUM 34a, GAME_ROWNUM 34a,

GET_BYTE_AND_SHIFT_FOR_COL 31b, GET_SCREEN_COORDS_FOR 31a, HGR_PAGE 28b, MASKO 34a,

MASK1 34a, PIXEL_MASKO 34b, PIXEL_MASK1 34b, ROWNUM 34a, ROW_ADDR 28b, ROW_COUNT 25c,

ROW_TO_ADDR 28c, SCREEN_ROW_TABLE 30, and SPRITE_NUM 25c.

There is a different routine which erases a sprite at a given screen coordinate. It does this by drawing the inverse of the sprite on page 1, then drawing the sprite data from page 2 (the background page) onto page 1.

Upon entry, the Y register needs to be set to the screen row coordinate (0-191). However, the X register needs to be set to half the screen column coordinate (0-139) because otherwise the maximum coordinate (279) wouldn't fit in a register.

```

38  <erase sprite at screen coordinate 38>≡ (278)
      ORG      $8336
      ERASE_SPRITE_AT_PIXEL_COORDS:
      SUBROUTINE
      ; Enter routine with A set to sprite number to draw,
      ; Y set to the screen row to erase it at, and X
      ; set to *half* the screen column to erase it at.

      STY      ROWNUM
      STA      SPRITE_NUM
      JSR      GET_BYTE_AND_SHIFT_FOR_HALF_SCREEN_COL
      STA      COLNUM
      STX      COL_SHIFT_AMT
      JSR      COMPUTE_SHIFTED_SPRITE

      LDX      #$0B
      STX      ROW_COUNT
      LDX      #$00
      LDA      COL_SHIFT_AMT
      CMP      #$05
      BCS      .need_3_bytes      ; If COL_SHIFT_AMT >= 5, we need to alter three
                                   ; screen bytes, otherwise just two bytes.

      .loop1:
      LDY      ROWNUM
      JSR      ROW_TO_ADDR_FOR_BOTH_PAGES
      LDY      COLNUM
      LDA      BLOCK_DATA,X
      EOR      #$7F
      AND      (ROW_ADDR),Y
      ORA      (ROW_ADDR2),Y
      STA      (ROW_ADDR),Y      ; screen[COLNUM] =
                                   ; (screen[COLNUM] & (BLOCK_DATA[i] ^ 0x7F)) |
                                   ; screen2[COLNUM]

      INX      ; X++
      INY      ; Y++
      LDA      BLOCK_DATA,X
      EOR      #$7F
      AND      (ROW_ADDR),Y
      ORA      (ROW_ADDR2),Y
      STA      (ROW_ADDR),Y      ; screen[COLNUM+1] =

```

```

; (screen[COLNUM+1] & (BLOCK_DATA[i+1] ^ 0x7F)) |
; screen2[COLNUM+1]

    INX                ; X++
    INX                ; X++
    INC    ROWNUM
    DEC    ROW_COUNT
    BNE    .loop1
    RTS

.need_3_bytes:
    LDY    ROWNUM
    JSR    ROW_TO_ADDR_FOR_BOTH_PAGES
    LDY    COLNUM
    LDA    BLOCK_DATA,X
    EOR    #$7F
    AND    (ROW_ADDR),Y
    ORA    (ROW_ADDR2),Y
    STA    (ROW_ADDR),Y        ; screen[COLNUM] =
                                ; (screen[COLNUM] & (BLOCK_DATA[i] ^ 0x7F)) |
                                ; screen2[COLNUM]

    INX                ; X++
    INY                ; Y++
    LDA    BLOCK_DATA,X
    EOR    #$7F
    AND    (ROW_ADDR),Y
    ORA    (ROW_ADDR2),Y
    STA    (ROW_ADDR),Y        ; screen[COLNUM+1] =
                                ; (screen[COLNUM+1] & (BLOCK_DATA[i+1] ^ 0x7F)) |
                                ; screen2[COLNUM+1]

    INX                ; X++
    INY                ; Y++
    LDA    BLOCK_DATA,X
    EOR    #$7F
    AND    (ROW_ADDR),Y
    ORA    (ROW_ADDR2),Y
    STA    (ROW_ADDR),Y        ; screen[COLNUM+2] =
                                ; (screen[COLNUM+2] & (BLOCK_DATA[i+2] ^ 0x7F)) |
                                ; screen2[COLNUM+2]

    INX                ; X++
    INC    ROWNUM
    DEC    ROW_COUNT
    BNE    .need_3_bytes
    RTS

```

Defines:

ERASE_SPRITE_AT_PIXEL_COORDS, used in chunks 126, 137, 157, 159, 161, 164, 168, 171, 175, 184, 193, 204, 206, 208, and 210.

Uses BLOCK_DATA 23, COLNUM 34a, COL_SHIFT_AMT 34a, COMPUTE_SHIFTED_SPRITE 26,
 GET_BYTE_AND_SHIFT_FOR_HALF_SCREEN_COL 32a, ROWNUM 34a, ROW_ADDR 28b, ROW_ADDR2 28b,
 ROW_COUNT 25c, ROW_TO_ADDR_FOR_BOTH_PAGES 29a, and SPRITE_NUM 25c.

And then there's the corresponding routine to draw a sprite at the given co-
 ordinates. The routine also sets whether the active and the background screens
 differ in SCREENS_DIFFER.

```
40  <defines 4>+≡ (281) <34a 45>
      SCREENS_DIFFER EQU $52
```

Defines:

SCREENS_DIFFER, used in chunks 41 and 43.

```

41  <draw sprite at screen coordinate 41>≡ (278)
      ORG      $83A7
      DRAW_SPRITE_AT_PIXEL_COORDS:
      SUBROUTINE
      ; Enter routine with A set to sprite number to draw,
      ; Y set to the screen row to draw it at, and X
      ; set to *half* the screen column to draw it at.

      STY      ROWNUM
      STA      SPRITE_NUM
      JSR      GET_BYTE_AND_SHIFT_FOR_HALF_SCREEN_COL
      STA      COLNUM
      STX      COL_SHIFT_AMT
      JSR      COMPUTE_SHIFTED_SPRITE

      LDA      #$0B
      STA      ROW_COUNT
      LDX      #$00
      STX      SCREENS_DIFFER      ; SCREENS_DIFFER = 0
      LDA      COL_SHIFT_AMT
      CMP      #$05
      BCS      .need_3_bytes      ; If COL_SHIFT_AMT >= 5, we need to alter three
                                   ; screen bytes, otherwise just two bytes.

      .loop1:
      LDY      ROWNUM
      JSR      ROW_TO_ADDR_FOR_BOTH_PAGES
      LDY      COLNUM
      LDA      (ROW_ADDR),Y
      EOR      (ROW_ADDR2),Y
      AND      BLOCK_DATA,X
      ORA      SCREENS_DIFFER
      STA      SCREENS_DIFFER      ; SCREENS_DIFFER |=
                                   ; ( (screen[COLNUM] ^ screen2[COLNUM]) &
                                   ;   BLOCK_DATA[i] )

      LDA      BLOCK_DATA,X
      ORA      (ROW_ADDR),Y
      STA      (ROW_ADDR),Y      ; screen[COLNUM] |= BLOCK_DATA[i]

      INX      ; X++
      INY      ; Y++
      LDA      (ROW_ADDR),Y
      EOR      (ROW_ADDR2),Y
      AND      BLOCK_DATA,X
      ORA      SCREENS_DIFFER
      STA      SCREENS_DIFFER      ; SCREENS_DIFFER |=
                                   ; ( (screen[COLNUM+1] ^ screen2[COLNUM+1]) &
                                   ;   BLOCK_DATA[i+1] )

      LDA      BLOCK_DATA,X
      ORA      (ROW_ADDR),Y

```

```

        STA      (ROW_ADDR),Y          ; screen[COLNUM+1] |= BLOCK_DATA[i+1]

        INX
        INX                          ; X++
        INC      ROWNUM
        DEC      ROW_COUNT
        BNE      .loop1
        RTS

.needs_3_bytes:
        LDY      ROWNUM
        JSR      ROW_TO_ADDR_FOR_BOTH_PAGES
        LDY      COLNUM
        LDA      (ROW_ADDR),Y
        EOR      (ROW_ADDR2),Y
        AND      BLOCK_DATA,X
        ORA      SCREENS_DIFFER
        STA      SCREENS_DIFFER        ; SCREENS_DIFFER |=
                                        ; (screen[COLNUM] ^ screen2[COLNUM]) &
                                        ; BLOCK_DATA[i])

        LDA      BLOCK_DATA,X
        ORA      (ROW_ADDR),Y
        STA      (ROW_ADDR),Y          ; screen[COLNUM] |= BLOCK_DATA[i]

        INX
        INY                          ; X++
                                        ; Y++
        LDA      (ROW_ADDR),Y
        EOR      (ROW_ADDR2),Y
        AND      BLOCK_DATA,X
        ORA      SCREENS_DIFFER
        STA      SCREENS_DIFFER        ; SCREENS_DIFFER |=
                                        ; (screen[COLNUM+1] ^ screen2[COLNUM+1]) &
                                        ; BLOCK_DATA[i+1])

        LDA      BLOCK_DATA,X
        ORA      (ROW_ADDR),Y
        STA      (ROW_ADDR),Y          ; screen[COLNUM+1] |= BLOCK_DATA[i+1]

        INX
        INY                          ; X++
                                        ; Y++
        LDA      (ROW_ADDR),Y
        EOR      (ROW_ADDR2),Y
        AND      BLOCK_DATA,X
        ORA      SCREENS_DIFFER
        STA      SCREENS_DIFFER        ; SCREENS_DIFFER |=
                                        ; (screen[COLNUM+2] ^ screen2[COLNUM+2]) &
                                        ; BLOCK_DATA[i+2])

        LDA      BLOCK_DATA,X
        ORA      (ROW_ADDR),Y
        STA      (ROW_ADDR),Y          ; screen[COLNUM+2] |= BLOCK_DATA[i+2]

```

```

        INX                      ; X++
        INC     ROWNUM
        DEC     ROW_COUNT
        BNE     .need_3_bytes
        RTS

```

Defines:

DRAW_SPRITE_AT_PIXEL_COORDS, used in chunks 43, 168, 171, 179, 188, 193, 204, 206, 208, and 212.

Uses BLOCK_DATA 23, COLNUM 34a, COL_SHIFT_AMT 34a, COMPUTE_SHIFTED_SPRITE 26, GET_BYTE_AND_SHIFT_FOR_HALF_SCREEN_COL 32a, ROWNUM 34a, ROW_ADDR 28b, ROW_ADDR2 28b, ROW_COUNT 25c, ROW_TO_ADDR_FOR_BOTH_PAGES 29a, SCREENS_DIFFER 40, and SPRITE_NUM 25c.

There is a special routine to draw the player sprite at the player's location. If the two pages at the player's location are different and the player didn't pick up gold (which would explain the difference), then the player is killed.

```

43  <draw player 43>≡ (278)
        ORG     $6C02
        DRAW_PLAYER:
        SUBROUTINE

        JSR     GET_SPRITE_AND_SCREEN_COORD_AT_PLAYER
        JSR     DRAW_SPRITE_AT_PIXEL_COORDS
        LDA     SCREENS_DIFFER
        BEQ     .end
        LDA     DIDNT_PICK_UP_GOLD
        BEQ     .end
        LSR     ALIVE      ; Set player as dead
        .end
        RTS

```

Defines:

DRAW_PLAYER, used in chunks 157, 161, 164, 168, 171, and 175.

Uses ALIVE 111a, DIDNT_PICK_UP_GOLD 136b, DRAW_SPRITE_AT_PIXEL_COORDS 41, GET_SPRITE_AND_SCREEN_COORD_AT_PLAYER 135b, and SCREENS_DIFFER 40.

3.5 Printing strings

Now that we can put sprites onto the screen at any game coordinate, we can also have some routines that print strings. We saw above that we have letter and number sprites, plus some punctuation. Letters and punctuation are always blue, while numbers are always orange.

There is a basic routine to put a character at the current `GAME.COLNUM` and `GAME.ROWNUM`, incrementing this "cursor", and putting it at the beginning of the next line if we "print" a newline character.

We first define a routine to convert the ASCII code of a character to its sprite number. Lode Runner sets the high bit of the code to make it be treated as ASCII.

```

44  <char to sprite num 44>≡ (278)
      ORG      $7B2A
      CHAR_TO_SPRITE_NUM:
      SUBROUTINE
      ; Enter routine with A set to the ASCII code of the
      ; character to convert to sprite number, with the high bit set.
      ; The sprite number is returned in A.

      CMP      #$C1                ; 'A' -> sprite 69
      BCC      .not_letter
      CMP      #$DB                ; 'Z' -> sprite 94
      BCC      .letter

      .not_letter:
      ; On return, we will subtract 0x7C from X to
      ; get the actual sprite. This is to make A-Z
      ; easier to handle.
      LDX      #$7C
      CMP      #$A0                ; ' ' -> sprite 0
      BEQ      .end
      LDX      #$DB
      CMP      #$BE                ; '>' -> sprite 95
      BEQ      .end
      INX
      CMP      #$AE                ; '.' -> sprite 96
      BEQ      .end
      INX
      CMP      #$A8                ; '(' -> sprite 97
      BEQ      .end
      INX
      CMP      #$A9                ; ')' -> sprite 98
      BEQ      .end
      INX
      CMP      #$AF                ; '/' -> sprite 99
      BEQ      .end
      INX
      CMP      #$AD                ; '-' -> sprite 100

```



```

        BEQ      .end
        INX
        CMP      #$BC                      ; '<' -> sprite 101
        BEQ      .end
        LDA      #$10                      ; sprite 16: just one of the man sprites
        RTS

.end:
        TXA

.letter:
        SEC
        SBC      #$7C
        RTS

```

Defines:

CHAR.TO.SPRITE_NUM, used in chunks 46a and 233.

Now we can define the routine to put a character on the screen at the current position.

```

45  <defines 4>+≡ (281) <40 46b>
        DRAW_PAGE EQU      $87          ; 0x20 for page 1, 0x40 for page 2

```

Defines:

DRAW_PAGE, used in chunks 46a, 48a, 53, 119b, 123, 124, 233, 237, 259, 265, and 268.

```

46a  <put char 46a>≡ (278)
      ORG      $7B64
      PUT_CHAR:
      SUBROUTINE
      ; Enter routine with A set to the ASCII code of the
      ; character to put on the screen, with the high bit set.

      CMP      #$8D
      BEQ      NEWLINE          ; If newline, do NEWLINE instead.
      JSR      CHAR_TO_SPRITE_NUM
      LDY      DRAW_PAGE
      CPY      #$40
      BEQ      .draw_to_page2

      JSR      DRAW_SPRITE_PAGE1
      INC      GAME_COLNUM
      RTS

      .draw_to_page2
      JSR      DRAW_SPRITE_PAGE2
      INC      GAME_COLNUM
      RTS

      NEWLINE:
      SUBROUTINE
      INC      GAME_ROWNUM
      LDA      #$00
      STA      GAME_COLNUM
      RTS

```

Defines:

NEWLINE, used in chunk 122b.

PUT_CHAR, used in chunks 47, 120c, 121b, and 233.

Uses CHAR_TO_SPRITE_NUM 44, DRAW_PAGE 45, DRAW_SPRITE_PAGE1 35, DRAW_SPRITE_PAGE2 35, GAME_COLNUM 34a, and GAME_ROWNUM 34a.

The PUT_STRING routine uses PUT_CHAR to put a string on the screen. Rather than take an address pointing to a string, instead it uses the return address as the source for data. It then has to fix up the actual return address at the end to be just after the zero-terminating byte of the string.

```

46b  <defines 4>+≡ (281) <45 48b>
      SAVED_RET_ADDR      EQU      $10      ; 2 bytes

```

Defines:

SAVED_RET_ADDR, used in chunks 47 and 58.

```

47  <put string 47>≡ (278)
      ORG      $86E0
      PUT_STRING:
      SUBROUTINE

      PLA
      STA      SAVED_RET_ADDR
      PLA
      STA      SAVED_RET_ADDR+1
      BNE      .next

      .loop:
      LDY      #$00
      LDA      (SAVED_RET_ADDR),Y
      BEQ      .end
      JSR      PUT_CHAR

      .next:
      INC      SAVED_RET_ADDR
      BNE      .loop
      INC      SAVED_RET_ADDR+1
      BNE      .loop

      .end:
      LDA      SAVED_RET_ADDR+1
      PHA
      LDA      SAVED_RET_ADDR
      PHA
      RTS

```

Defines:

PUT_STRING, used in chunks 53, 73a, 120, 121, 237, 241, 244, 259, 261–64, and 268.
 Uses PUT_CHAR 46a and SAVED_RET_ADDR 46b.

Like PUT_CHAR, we also have PUT_DIGIT which draws the sprite corresponding to digits 0 to 9 at the current position, incrementing the cursor.

```

48a  <put digit 48a>≡ (278)
      ORG      $7B15
      PUT_DIGIT:
      SUBROUTINE
      ; Enter routine with A set to the digit to put on the screen.

      CLC
      ADC      #$3B                ; '0' -> sprite 59, '9' -> sprite 68.
      LDX      DRAW_PAGE
      CPX      #$40
      BEQ      .draw_to_page2
      JSR      DRAW_SPRITE_PAGE1
      INC      GAME_COLNUM
      RTS

      .draw_to_page2:
      JSR      DRAW_SPRITE_PAGE2
      INC      GAME_COLNUM
      RTS

```

Defines:

PUT_DIGIT, used in chunks 51, 53, 74, and 120–22.

Uses DRAW_PAGE 45, DRAW_SPRITE_PAGE1 35, DRAW_SPRITE_PAGE2 35, and GAME_COLNUM 34a.

3.6 Numbers

We also need a way to put numbers on the screen.

First, a routine to convert a one-byte decimal number into hundreds, tens, and units.

```

48b  <defines 4>+≡ (281) <46b 50b>
      HUNDREDS    EQU    $89
      TENS        EQU    $8A
      UNITS       EQU    $8B

```

Defines:

HUNDREDS, used in chunks 49, 53, 74, and 121c.

TENS, used in chunks 49–51, 53, 74, 121c, and 122a.

UNITS, used in chunks 49–51, 53, 74, 121c, and 122a.

```
49  <to decimal3 49>≡ (278)
      ORG      $7AF8
      TO_DECIMAL3:
      SUBROUTINE
      ; Enter routine with A set to the number to convert.

      LDX      #$00
      STX      TENS
      STX      HUNDREDS

      .loop1:
      CMP      #100
      BCC      .loop2
      INC      HUNDREDS
      SBC      #100
      BNE      .loop1

      .loop2:
      CMP      #10
      BCC      .end
      INC      TENS
      SBC      #10
      BNE      .loop2

      .end:
      STA      UNITS
      RTS
```

Defines:

TO_DECIMAL3, used in chunks 53, 74, and 121c.
Uses HUNDREDS 48b, TENS 48b, and UNITS 48b.

There's also a routine to convert a BCD byte to tens and units.

50a $\langle bcd\ to\ decimal2\ 50a \rangle \equiv$ (278)

```

    ORG      $7AE9
BCD_TO_DECIMAL2:
    SUBROUTINE
    ; Enter routine with A set to the BCD number to convert.

    STA      TENS
    AND      #$0F
    STA      UNITS
    LDA      TENS
    LSR
    LSR
    LSR
    LSR
    STA      TENS
    RTS

```

Defines:

BCD_TO_DECIMAL2, used in chunks 51 and 122a.
 Uses TENS 48b and UNITS 48b.

3.7 Score and status

Lode Runner stores your score as an 8-digit BCD number.

50b $\langle defines\ 4 \rangle + \equiv$ (281) $\triangleleft 48b\ 52 \triangleright$

```

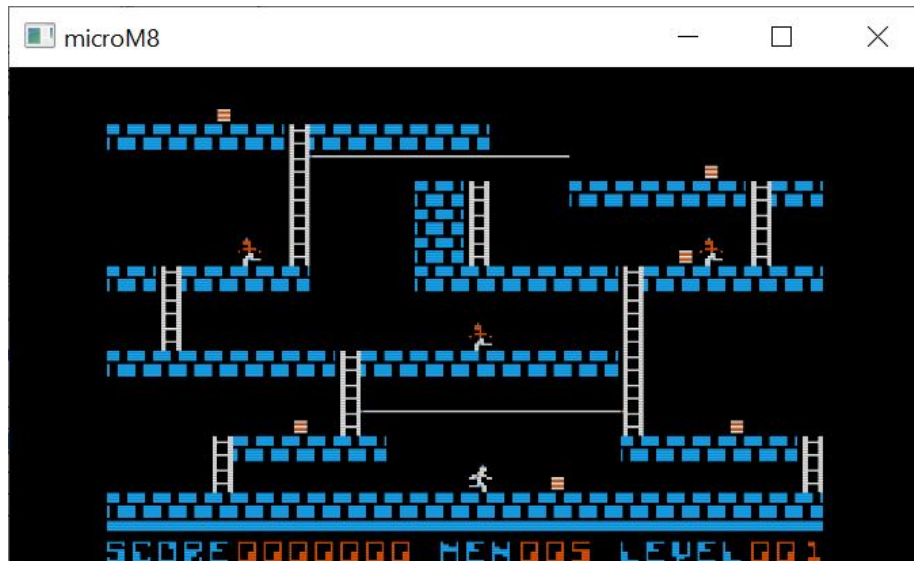
    SCORE    EQU      $8E      ; 4 bytes, BCD format, tens/units in first byte.

```

Defines:

SCORE, used in chunks 51, 53, 120a, 126, 137, 193, 233, 244, 246, 250, and 259.

The score is always put on the screen at row 16 column 5, but only the last 7 digits. Row 16 is the status line, as can be seen at the bottom of this screenshot.



There's a routine to add a 4-digit BCD number to the score and then update it on the screen.

```

51  <add and update score 51>≡ (278)
      ORG      $7A92
      ADD_AND_UPDATE_SCORE:
      SUBROUTINE
      ; Enter routine with A set to BCD tens/units and
      ; Y set to BCD thousands/hundreds.

      CLC
      SED                      ; Turn on BCD addition mode.
      ADC      SCORE
      STA      SCORE
      TYA
      ADC      SCORE+1
      STA      SCORE+1
      LDA      #$00
      ADC      SCORE+2
      STA      SCORE+2
      LDA      #$00
      ADC      SCORE+3
      STA      SCORE+3          ; SCORE += param
      CLD                      ; Turn off BCD addition mode.

      LDA      #5
      STA      GAME_COLNUM

```

```

LDA    #16
STA    GAME_ROWNUM

LDA    SCORE+3
JSR    BCD_TO_DECIMAL2
LDA    UNITS          ; Note we skipped TENS.
JSR    PUT_DIGIT

LDA    SCORE+2
JSR    BCD_TO_DECIMAL2
LDA    TENS
JSR    PUT_DIGIT
LDA    UNITS
JSR    PUT_DIGIT

LDA    SCORE+1
JSR    BCD_TO_DECIMAL2
LDA    TENS
JSR    PUT_DIGIT
LDA    UNITS
JSR    PUT_DIGIT

LDA    SCORE
JSR    BCD_TO_DECIMAL2
LDA    TENS
JSR    PUT_DIGIT
LDA    UNITS
JMP    PUT_DIGIT          ; tail call

```

Defines:

ADD_AND_UPDATE_SCORE, used in chunks 53, 126, 137, 193, and 250.

Uses BCD_TO_DECIMAL2 50a, GAME_COLNUM 34a, GAME_ROWNUM 34a, PUT_DIGIT 48a, SCORE 50b, TENS 48b, and UNITS 48b.

The other elements in the status line are the number of men (i.e. lives) and the current level.

```

52  <defines 4>+≡ (281) <50b 57>
      LEVELNUM    EQU    $A6
      LIVES       EQU    $98

```

Defines:

LEVELNUM, used in chunks 53, 74, 109b, 132b, 133c, 140a, 233, 250, and 268.

LIVES, used in chunks 53, 140, 141, 148, 246, 250, and 259.

Here are the routines to put the lives and level number on the status line.
Lives starts at column 16, and level number starts at column 25.

```

53  <put status 53>≡ (278)
      ORG      $7A70
      PUT_STATUS_LIVES:
      SUBROUTINE

      LDA      LIVES
      LDX      #16
      ; fallthrough

      PUT_STATUS_BYTE:
      SUBROUTINE
      ; Puts the number in A as a three-digit decimal on the screen
      ; at row 16, column X.

      STX      GAME_COLNUM
      JSR      TO_DECIMAL3
      LDA      #16
      STA      GAME_ROWNUM
      LDA      HUNDREDS
      JSR      PUT_DIGIT
      LDA      TENS
      JSR      PUT_DIGIT
      LDA      UNITS
      JMP      PUT_DIGIT          ; tail call

      PUT_STATUS_LEVEL:
      SUBROUTINE

      LDA      LEVELNUM
      LDX      #25
      BNE      PUT_STATUS_BYTE    ; Unconditional jump

      ORG      $79AD
      PUT_STATUS:
      SUBROUTINE

      JSR      CLEAR_HGR1
      JSR      CLEAR_HGR2

      PUT_STATUS_DRAW:
      LDY      #$27
      LDA      DRAW_PAGE
      CMP      #$40
      BEQ      .draw_line_on_page_2

      .draw_line_on_page_1:
      LDA      #$AA

```

```

        STA      $2350,Y
        STA      $2750,Y
        STA      $2B50,Y
        STA      $2F50,Y
        DEY
        LDA      #$D5
        STA      $2350,Y
        STA      $2750,Y
        STA      $2B50,Y
        STA      $2F50,Y
        DEY
        BPL      .draw_line_on_page_1
        BMI      .end          ; Unconditional

.draw_line_on_page_2:
        LDA      #$AA
        STA      $4350,Y
        STA      $4750,Y
        STA      $4B50,Y
        STA      $4F50,Y
        DEY
        LDA      #$D5
        STA      $4350,Y
        STA      $4750,Y
        STA      $4B50,Y
        STA      $4F50,Y
        DEY
        BPL      .draw_line_on_page_2

.end:
        LDA      #$10
        STA      GAME_ROWNUM
        LDA      #$00
        STA      GAME_COLNUM

        ; "SCORE      MEN      LEVEL      "
        JSR      PUT_STRING
        HEX      D3 C3 CF D2 C5 A0 A0 A0 A0 A0 A0 A0 A0 CD C5 CE
        HEX      A0 A0 A0 A0 CC C5 D6 C5 CC A0 A0 A0 00

        JSR      PUT_STATUS_LIVES
        JSR      PUT_STATUS_LEVEL
        LDA      #$00
        TAY
        JMP      ADD_AND_UPDATE_SCORE      ; tailcall

```

Defines:

PUT_STATUS, used in chunk 246.
 PUT_STATUS_LEVEL, used in chunk 89.
 PUT_STATUS_LIVES, used in chunks 89, 140b, and 250.

July 8, 2026

main.nw 55

Uses ADD_AND_UPDATE_SCORE 51, CLEAR_HGR1 5, CLEAR_HGR2 5, DRAW_PAGE 45, GAME_COLNUM 34a, GAME_ROWNUM 34a, HUNDREDS 48b, LEVELNUM 52, LIVES 52, PUT_DIGIT 48a, PUT_STRING 47, SCORE 50b, TENS 48b, TO_DECIMAL3 49, and UNITS 48b.